DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

CPU Scheduling

CMSC 125 Lab 2

Prepared by: Rene Andre B. Jocsing

Published: February 26, 2026 **Deadline:** May 21, 2026

This laboratory assignment focuses on implementing CPU scheduling in C. You will build a discrete-event simulator that demonstrates how operating systems make scheduling decisions to optimize system performance. Through this exercise, you will gain hands-on experience with the algorithms covered in lectures and develop insight into the tradeoffs between different scheduling policies.

Background

CPU scheduling is one of the most fundamental responsibilities of an operating system. When multiple processes compete for CPU time, the scheduler must decide which process runs next and for how long. Different scheduling algorithms optimize for different metrics—some prioritize fairness, others minimize turnaround time or response time, and some attempt to balance multiple objectives simultaneously.

Your simulator will accept a workload specification (a set of processes with arrival times and burst times) and simulate how each scheduling algorithm would handle that workload. The simulator will compute key performance metrics and allow comparison between algorithms.

Learning Objectives

By the end of this laboratory exercise, students should be able to:

- Implement fundamental CPU scheduling algorithms: FCFS, SJF, STCF, and RR
- Design and implement a Multi-Level Feedback Queue (MLFQ) scheduler
- Calculate scheduling metrics: turnaround time, response time, and waiting time
- Analyze the performance characteristics and tradeoffs of different algorithms
- Work with discrete-event simulation and priority queues in C
- Generate and interpret Gantt charts from scheduling simulations
- Apply data structures (linked lists, queues, heaps) to systems problems
- Make informed design decisions and justify them with empirical evidence

---

Task

Build a CPU scheduling simulator (`schedsim`) that:

- Reads process workloads from input files or command-line arguments
- Implements four classic scheduling algorithms: FCFS, SJF, STCF, RR

- Implements a Multi-Level Feedback Queue of your own design
- Calculates and reports scheduling metrics for each algorithm
- Outputs Gantt charts showing process execution timelines
- Supports configurable parameters (time quantum for RR, MLFQ configuration)
- Provides comparative analysis across algorithms

Required Features

Input Format

Your simulator must accept workloads in the following format:

```
# Process workload specification
# Format: PID ArrivalTime BurstTime
A 0 240
B 10 180
C 20 150
D 25 80
E 30 130
```

Lines beginning with # are comments. Each process is specified by:

- **PID**: Process identifier (string or integer)
- **ArrivalTime**: Time when process enters the ready queue (integer)
- **BurstTime**: Total CPU time required (integer)

Important note on BurstTime: While your input format includes BurstTime for simulation purposes, your **MLFQ implementation must not read or use this value**. MLFQ must dynamically determine a process's behavior (short vs. long-running) through observation during execution. This is the fundamental difference between MLFQ and algorithms like SJF/STCF that require knowing burst times in advance. The BurstTime field is only used by your simulator to know when a process completes. But MLFQ scheduling decisions must be made without consulting this value.

Your simulator should also accept workloads via command-line arguments:

```
$ ./schedsim --algorithm=FCFS --processes="A:0:240,B:10:180,C:20:150"
```

Hence, it must also be callable as a compiled binary using your Unix Shell from Lab 1.

Algorithm Implementation

First-Come First-Serve (FCFS)

Implement non-preemptive FCFS scheduling:

- Processes execute in order of arrival
- Once a process starts, it runs to completion
- Simple queue-based implementation

```
$ ./schedsim --algorithm=FCFS --input=workload.txt
Running FCFS Scheduler...

=== Gantt Chart ===
[A--------------][B--------][C------]
```

```
Time: 0        240       420      570

=== Metrics ===
Process | AT   | BT   | FT   | TT   | WT   | RT
--------|------|------|------|------|------|------
A       |    0 | 240  | 240  | 240  |    0 |    0
B       |   10 | 180  | 420  | 410  | 230  | 230
C       |   20 | 150  | 570  | 550  | 400  | 400
--------|------|------|------|------|------|------
Average |      |      |      | 400  | 210  | 210

Convoy effect detected: Process B waited 230 time units
```

Shortest Job First (SJF)

Implement non-preemptive SJF:

- Select process with shortest burst time from ready queue
- Once selected, runs to completion
- Optimal for average turnaround time (non-preemptive case)
- Requires sorting or priority queue (which is better?)

Shortest Time-to-Completion First (STCF)

Implement preemptive SJF (STCF):

- Always run process with shortest remaining time
- Preempt current process if new arrival has shorter remaining time
- Optimal for average turnaround time (preemptive case)
- Track remaining burst time for each process

```
$ ./schedsim --algorithm=STCF --input=workload.txt

=== Gantt Chart ===
[A][C---][D--][E---][B----][A----------]
Time: 0 10   100  150   250     400    570

Process A was preempted at t=10 (remaining: 230)
Process A resumed at t=400
```

Round Robin (RR)

Implement preemptive round-robin scheduling:

- Each process gets fixed time quantum `q`
- After quantum expires, process moves to end of ready queue
- Fair allocation of CPU time
- Configurable time quantum (default: 30)
- Note that queues must run in isolation during quantums (recall: lecture nuance)

```
$ ./schedsim --algorithm=RR --quantum=50 --input=workload.txt

Using time quantum q=50

=== Gantt Chart ===
```

```
[A-][B-][C-][D-][E-][A-][B-][C-]...
Time: 0  50 100 150 200 250 300 350

Total context switches: 18
Average response time: 45.2
```

Multi-Level Feedback Queue (MLFQ)

Design and implement your own MLFQ scheduler with the following requirements:

- **Minimum 3 priority levels** (you may use more)
- **Configurable time quantum per level** (must decrease or stay constant at lower priorities)
- **Priority boost mechanism** after period S
- **Allotment tracking** to prevent gaming via yielding
- **Downward demotion** when process exhausts time allotment
- **No knowledge of BurstTime**: Your MLFQ must *not* read the BurstTime field from the input. It must infer whether a process is interactive (short) or compute-intensive (long) through observation alone. This is the key insight of MLFQ—it adapts to process behavior without requiring *a priori* knowledge.
- **Justification** of your design parameters in documentation

Example MLFQ configuration:

```
# Multi-Level Feedback Queue Configuration
# Format: QueueID TimeQuantum Allotment

# High priority (interactive jobs)
Q0 10 50

# Medium priority (mixed workloads)
Q1 30 150

# Low priority (batch jobs) - FCFS
Q2 -1 -1

# Priority boost period
BOOST_PERIOD 200
```

```
$ ./schedsim --algorithm=MLFQ --mlfq-config=mlfq_config.txt \
             --input=workload.txt

=== MLFQ Configuration ===
Queue 0: q=10, allotment=50 (highest priority)
Queue 1: q=30, allotment=150
Queue 2: FCFS (lowest priority)
Boost period: 200

=== Execution Trace ===
t=0:   Process A enters Q0
t=10:  Process A -> Q1 (exhausted Q0 allotment)
t=50:  Priority boost: all processes -> Q0
t=60:  Process B completes in Q0
...

=== Analysis ===
Interactive job (short burst) behavior:
  - Process D stayed in Q0 (completed in 80 time units)
  - Average response time: 5.0

```

```
Long-running job behavior:
  - Process A demoted to Q2 after 200 time units
  - Turnaround time: 570 (fair for its burst time)

Your MLFQ successfully balanced responsiveness and fairness.
```

Metrics Calculation

For each algorithm, compute and display:

- **Finish Time (FT)**: When process completes
- **Turnaround Time (TT)**: FT - ArrivalTime
- **Waiting Time (WT)**: Time spent in ready queue
- **Response Time (RT)**: Time until first execution from arrival time

Calculate both per-process and average metrics. Your output should clearly show the formula used:

```
Process A:
  Arrival Time:     0
  Burst Time:       240
  Finish Time:      240
  Turnaround Time:  240 - 0 = 240
  Waiting Time:     240 - 240 = 0
  Response Time:    0 - 0 = 0
```

Gantt Chart Generation

Generate ASCII Gantt charts showing process execution over time:

- Represent each time unit with a character
- Use different symbols/letters for different processes
- Show time markers at regular intervals
- Scale appropriately for long simulations

For long simulations, provide a scaled view:

```
Each char = 10 time units:
[AAAA][BBB][CCC][DD][EEE]
Time: 0   100 180 250 280 350
```

Comparative Analysis

Provide a comparison mode that runs all algorithms on the same workload:

```
$ ./schedsim --compare --input=workload.txt

=== Algorithm Comparison for workload.txt ===

Algorithm | Avg TT | Avg WT | Avg RT | Context Switches
----------|--------|--------|--------|-----------------
FCFS      |  400.0 |  210.0 |  210.0 |        0
SJF       |  340.0 |  150.0 |  150.0 |        0
STCF      |  280.0 |   90.0 |   25.0 |       12
RR (q=30) |  385.0 |  195.0 |   45.0 |       28
MLFQ      |  310.0 |  120.0 |   38.0 |       15
```

Technical Requirements

Process Execution Model

Important: Your simulator must be designed to run as a child process spawned via `fork()` and `exec()`. When invoked from the command line or by another program, it should function correctly as a standalone executable. This mimics how real operating systems launch processes, and you will be testing this during defense using your Unix Shell from Lab 1.

Example expected behavior:

```c
// From parent process (e.g., your shell or test harness)
pid_t pid = fork();
if (pid == 0) {
    // Child process
    char *args[] = {"./schedsim", "--algorithm=FCFS",
                    "--input=workload.txt", NULL};
    execvp(args[0], args);
    perror("exec failed");
    exit(1);
} else {
    // Parent waits for simulator to complete
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) {
        printf("Simulator exited with code %d\n", WEXITSTATUS(status));
    }
}
```

This design ensures your simulator:

- Functions correctly as a standalone executable
- Can be launched by other programs (including your Lab 1 shell)
- Returns appropriate exit codes (0 for success, non-zero for errors)
- Properly handles command-line arguments via `argv`
- Works with standard I/O streams (stdin, stdout, stderr)

Data Structures

Design appropriate data structures to represent processes and scheduling state:

```c
typedef struct {
    char pid[16];           // Process identifier
    int arrival_time;       // When process arrives
    int burst_time;         // Total CPU time needed
    int remaining_time;     // For preemptive algorithms
    int start_time;         // When first executed (for RT)
    int finish_time;        // When completed (for TT)
    int waiting_time;       // Time spent waiting
    int priority;           // For MLFQ
    int time_in_queue;      // For MLFQ allotment tracking
} Process;
```

```c
typedef struct {
    int level;              // Queue priority level (0 = highest)
    int time_quantum;       // Time slice for this queue (-1 for FCFS)
    int allotment;          // Max time before demotion (-1 for infinite)
    Process *queue;         // Array or linked list of processes
```

```
    int size;               // Current queue size
} MLFQQueue;

typedef struct {
    MLFQQueue *queues;      // Array of queues
    int num_queues;         // Number of priority levels
    int boost_period;       // Period for priority boost (S)
    int last_boost;         // Last boost time
} MLFQScheduler;
```

Scheduling Algorithms as Functions

Implement each algorithm as a separate function with a consistent interface. This ensures scalability should expansion of scheduler algorithms be required in the future:

```
typedef struct {
    Process *processes;     // Array of all processes
    int num_processes;      // Number of processes
    int current_time;       // Current simulation time
    // ... additional fields for metrics, Gantt chart, etc.
    // Recall: CMSC 141
} SchedulerState;

// Return 0 on success, -1 on error (command line etiquette)
int schedule_fcfs(SchedulerState *state);
int schedule_sjf(SchedulerState *state);
int schedule_stcf(SchedulerState *state);
int schedule_rr(SchedulerState *state, int quantum);
int schedule_mlfq(SchedulerState *state, MLFQConfig *config);
```

Simulation Engine

Implement a discrete-event simulation engine:

```
typedef enum {
    EVENT_ARRIVAL,
    EVENT_COMPLETION,
    EVENT_QUANTUM_EXPIRE,
    EVENT_PRIORITY_BOOST
} EventType;

typedef struct Event {
    int time;
    EventType type;
    Process *process;
    struct Event *next;
} Event;

void simulate_scheduler(SchedulerState *state,
                        SchedulingAlgorithm algorithm) {
    Event *event_queue = initialize_events(state);

    while (event_queue != NULL) {
        Event *current = pop_event(&event_queue);
        state->current_time = current->time;

        switch (current->type) {
            case EVENT_ARRIVAL:
```

```
                handle_arrival(state, current->process);
                break;
            case EVENT_COMPLETION:
                handle_completion(state, current->process);
                break;
            // ... handle other events
        }

        free(current);
    }

    calculate_metrics(state);
    print_results(state);
}
```

Code Organization

Structure your project with clear separation of concerns:

```
schedsim/
|-- Makefile
|-- README.md
|-- include/
|   |-- process.h           # Process data structures
|   |-- scheduler.h         # Scheduler interfaces
|   |-- metrics.h           # Metrics calculation
|   +-- gantt.h             # Gantt chart generation
|-- src/
|   |-- main.c              # CLI and main loop
|   |-- process.c           # Process management
|   |-- fcfs.c              # FCFS implementation
|   |-- sjf.c               # SJF implementation
|   |-- stcf.c              # STCF implementation
|   |-- rr.c                # Round Robin implementation
|   |-- mlfq.c              # MLFQ implementation
|   |-- metrics.c           # Metrics calculation
|   |-- gantt.c             # Gantt chart rendering
|   +-- utils.c             # Utility functions
|-- tests/
|   |-- workload1.txt       # Test workloads
|   |-- workload2.txt
|   +-- test_suite.sh       # Automated test script
+-- docs/
    +-- mlfq_design.md      # Your MLFQ design justification
```

Implementation Notes

Discrete-Event Simulation

Your simulator operates in discrete time steps. At each time unit:

1. Check for process arrivals
2. Update running process (decrement remaining time)
3. Handle process completion or quantum expiration
4. Select next process according to algorithm
5. Record state for Gantt chart

```c
for (int t = 0; !all_complete(processes); t++) {
    // Handle arrivals at time t
    for (int i = 0; i < num_processes; i++) {
        if (processes[i].arrival_time == t) {
            enqueue_ready(&ready_queue, &processes[i]);
            if (processes[i].start_time == -1) {
                processes[i].start_time = t;  // For response time
            }
        }
    }

    // Run current process for 1 time unit
    if (current_process != NULL) {
        current_process->remaining_time--;
        gantt_chart[t] = current_process->pid;

        if (current_process->remaining_time == 0) {
            current_process->finish_time = t + 1;
            current_process = NULL;
        }
    }

    // Select next process (algorithm-specific)
    if (current_process == NULL && !is_empty(ready_queue)) {
        current_process = select_next_process(&ready_queue);
    }
}
```

Metrics Calculation

Calculate metrics after simulation completes:

```c
void calculate_metrics(Process *processes, int n) {
    for (int i = 0; i < n; i++) {
        Process *p = &processes[i];

        // Turnaround time = Finish time - Arrival time
        p->turnaround_time = p->finish_time - p->arrival_time;

        // Waiting time = Turnaround time - Burst time
        p->waiting_time = p->turnaround_time - p->burst_time;

        // Response time = Start time - Arrival time
        p->response_time = p->start_time - p->arrival_time;
    }
}

double calculate_average_turnaround(Process *processes, int n) {
    double sum = 0.0;
    for (int i = 0; i < n; i++) {
        sum += processes[i].turnaround_time;
    }
    return sum / n;
}
```

MLFQ Implementation Strategy

For your MLFQ implementation, consider this approach:

1. New processes enter the highest-priority queue (Q0)
2. Each queue has its own time quantum and allotment
3. Track how much time a process has used in its current queue
4. When allotment is exhausted, demote to next lower queue
5. Implement priority boost by moving all processes to Q0 after period S (ensure processes cannot game the system by implementing T, maximum time in each queue, for better accounting)
6. Use essentially FCFS for the lowest queue (infinite time quantum)

```c
void mlfq_adjust_priority(MLFQScheduler *scheduler, Process *p) {
    MLFQQueue *current_queue = &scheduler->queues[p->priority];

    // Check if process exhausted its allotment
    if (p->time_in_queue >= current_queue->allotment) {
        // Demote to lower priority
        if (p->priority < scheduler->num_queues - 1) {
            remove_from_queue(current_queue, p);
            p->priority++;
            p->time_in_queue = 0;  // Reset allotment
            add_to_queue(&scheduler->queues[p->priority], p);
        }
    }
}

void mlfq_priority_boost(MLFQScheduler *scheduler, int current_time) {
    if (current_time - scheduler->last_boost >= scheduler->boost_period) {
        // Move all processes to highest priority
        for (int i = 1; i < scheduler->num_queues; i++) {
            MLFQQueue *queue = &scheduler->queues[i];
            while (queue->size > 0) {
                Process *p = dequeue(queue);
                p->priority = 0;
                p->time_in_queue = 0;
                enqueue(&scheduler->queues[0], p);
            }
        }
        scheduler->last_boost = current_time;
    }
}
```

Design Justification for MLFQ

Your `docs/mlfq_design.md` must justify your MLFQ parameters:

- **Number of queues**: Why did you choose 3, 4, or more levels?
- **Time quantum per level**: How did you decide quantum sizes?
- **Maximum time per level**: How did you decide the time a process is allowed on each queue?
- **Boost period**: How often should priority boosts occur?
- **Allotment values**: What reasoning led to your allotment choices?
- **Empirical testing**: Show test results supporting your design

Example justification:

> "We chose 4 priority levels because our test workloads showed a clear distinction between very short interactive jobs (<50 time units), medium-length jobs (50-200), long batch jobs (200-500), and very long computations (>500). Using quantum sizes of 10, 30, 100, and FCFS allowed short jobs to complete in Q0-Q1 with minimal context switching, while preventing long jobs from monopolizing the CPU. A boost period of 300 was selected after testing showed this prevented starvation while maintaining good responsiveness for new arrivals."

Common Pitfalls

- **Off-by-one errors:** Be careful with time boundaries. Does a process that arrives at t=10 and runs for 5 units complete at t=15 or t=14?
- **Start time tracking:** Response time requires knowing when a process *first* executes, not when it arrives. Track this separately.
- **STCF preemption:** When a new process arrives with shorter remaining time, preemption should happen immediately, not after current quantum.
- **RR quantum expiration:** Process moves to end of queue even if it has <q remaining time. Don't give it another quantum if it's about to complete.
- **MLFQ allotment tracking:** Allotment persists across quantum expirations within the same queue. Reset only on demotion.
- **MLFQ using BurstTime:** Your MLFQ implementation must NOT read or use the BurstTime field. It should only track remaining_time (decremented as the process runs) to know when a process completes. All scheduling decisions must be based on observed behavior (time in queue, allotment exhaustion) not on knowing the total burst time upfront.
- **Priority boost timing:** Boost should happen at regular intervals, not triggered by specific process events.
- **Waiting time calculation:** Waiting time = (Turnaround time) - (Burst time), not just time in ready queue. Be careful with context switches.
- **Memory management:** Free all dynamically allocated structures (queues, event lists, Gantt chart data).

Testing Strategy

Unit Testing

Test individual components:

- **Process arrival:** Verify processes enter ready queue at correct time
- **Queue operations:** Test enqueue, dequeue, sorting for SJF/STCF
- **Metrics calculation:** Hand-calculate expected values for small workloads
- **MLFQ transitions:** Test priority changes, boosts, allotment exhaustion

Algorithm Verification

Use the workloads from lecture quizzes to verify correctness:

```
# Quiz 3 workload from lesson3.md
# Expected results from lecture
A 0 240
B 10 180
C 20 150
D 25 80
E 30 130

# FCFS average TT should be 515
# SJF average TT should be 461
# STCF average TT should be 393
# RR (q=30) average TT should be 627
```

If your results don't match the lecture examples, your implementation has bugs (or the quiz has typos).

Edge Cases

- **Single process:** All algorithms should behave identically
- **Simultaneous arrivals:** Test tie-breaking (alphabetical by PID?)
- **Zero-time processes:** Process with burst time = 0 (should complete immediately)
- **Very long workloads:** Test with 100+ processes, ensure no memory issues
- **Identical burst times:** All processes have same burst time (SJF becomes FCFS)
- **Staircase arrivals:** Processes arrive at t = 0, 1, 2, 3, …

Performance Analysis

Test your MLFQ against different workload patterns:

- **All short jobs:** Most processes have burst time < 50
- **All long jobs:** Most processes have burst time > 200
- **Mixed workload:** Half short, half long
- **Bimodal:** Many very short jobs and a few very long jobs
- **I/O-bound simulation:** Short CPU bursts with gaps (simulate I/O)

For each workload, analyze:

- Average turnaround time vs. optimal (SJF or STCF)
- Average response time vs. RR
- Fairness: variance in waiting times
- Queue distribution: what % of time spent in each queue?

Automated Testing

Create a test script that verifies all algorithms:

```
#!/bin/bash
# test_suite.sh

echo "Running CMSC 125 Lab 2 Test Suite..."

# Test 1: Verify against lecture examples
echo "Test 1: Lecture Quiz 3 Workload"
./schedsim --algorithm=FCFS --input=tests/quiz3.txt > /tmp/fcfs.txt
if grep -q "Average.*515" /tmp/fcfs.txt; then
    echo "  FCFS: PASS"
else
    echo "  FCFS: FAIL (expected avg TT = 515)"
fi

# Test 2: Edge cases
echo "Test 2: Edge Cases"
./schedsim --algorithm=STCF --processes="A:0:0" > /dev/null
if [ $? -eq 0 ]; then
    echo "  Zero burst time: PASS"
else
    echo "  Zero burst time: FAIL"
fi

echo "Test suite complete."
echo "Note: Memory leaks will be checked via code review during defense."
```

Deliverables

Your group's GitHub repository, in addition to a clean, incremental commit history showing individual commits, must contain:

1. All source files (`.c` and `.h` files) with proper documentation
2. `Makefile` with targets:
   - `all`: Compile the simulator
   - `clean`: Remove binaries and object files
   - `test`: Run automated test suite
3. `README.md` with:
   - **Complete names** of group members
   - Compilation and usage instructions
   - List of implemented features and algorithms
   - Example usage commands with expected output
   - Known limitations or assumptions
4. `docs/mlfq_design.md` containing:
   - Detailed justification of your MLFQ design
   - Parameter choices (number of queues, quantums, allotments, boost period)
   - Empirical testing results supporting your design
   - Comparison with standard 3-level MLFQ
   - Discussion of tradeoffs in your design
5. `tests/` directory with:
   - Test workload files (including lecture examples)
   - Automated test script
   - Expected output files for verification
6. Screenshots or output logs demonstrating:
   - Each algorithm running successfully
   - Gantt charts for different workloads
   - Comparative analysis across algorithms
   - Your MLFQ handling different workload types

To submit, simply invite the instructor as a collaborator. Commits after the instructor has already graded the repository will not be considered.

Then, submit the following, **individually**, through email:

1. A short `reflection.txt` containing a brief summary of lessons learned, challenges encountered during the activity, and insights gained about CPU scheduling.
2. If in a group: a short `peer.txt` containing your thoughts about the work and conduct of your peers in the group during the activity.
3. The link to your GitHub repository (for verification)

Adhere to the following subject line: `[CMSC 125 Lab] Lab 2: Surname, Initials.`

For example: `[CMSC 125 Lab] Lab 2: Sanchez, SM.`

Academic Honesty

The usage of Large Language Models (e.g. ChatGPT, Claude, Deepseek, etc.) to generate code is considered cheating. As cheating is against the university's code of ethics, it is subject to failure in the course and harsh disciplinary action.

Important Dates

Progress reports and laboratory defense may be booked only during the dates and hours defined in the syllabus. It is mandatory to book appointments ahead of time on the course's booking page (also accessible on the course site).

Activity	Monday	Wednesday	Thursday
Week 1 Progress Report	Mar 2	Mar 4	Mar 5
Week 2 Progress Report	Mar 9	Mar 11	Mar 12
Week 3 Progress Report	Mar 16	Mar 18	Mar 19
Week 4 Laboratory Defense	**Mar 23**	**Mar 25**	**Mar 26**

The instructor must verify appointments ahead of time before they can be considered valid. See the syllabus for proper hours and more details.

Grading Rubric

Criteria	Excellent (90–100%)	Good (75–89%)	Fair (60–74%)	Poor (0–59%)
System Architecture (25%)	Modular design with clean separation of concerns; robust data structures; efficient resource usage.	Mostly modular; logic is functional but slightly coupled; data structures are appropriate.	Significant logic sprawl; monolithic functions; inconsistent data handling or hard-coded limits.	Spaghetti code; no modularity; violates basic systems programming principles.
Robustness (20%)	Handles complex edge cases and race conditions; perfect resource lifecycle; graceful error recovery.	Handles core features well; minor issues with fringe edge cases or synchronization logic.	Basic features work, but system is unstable; frequent resource leaks or intermittent errors.	Fails core logic; program crashes on unexpected input; incorrect usage of fundamental syscalls.
Code Engineering (10%)	No memory leaks; all syscalls check return codes; uses `perror` appropriately; safe pointer usage.	Minor leaks or missing checks on non-critical syscalls; mostly safe pointer arithmetic.	Inconsistent error handling; frequent unsafe operations; significant leaks or lack of bounds checking.	Frequent segmentation faults; silent failures of syscalls; no evidence of memory management.
Collaboration (20%)	Professional Git usage: atomic, semantic commits; use of feature branches; evidence of team collaboration.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history or shows no evidence of teamwork.
Technical Defense (25%)	Both members articulate the low-level mechanics; handles what-if scenarios and code-tracing confidently.	Clear architectural explanation; both members participate meaningfully; logic delivery is sound.	Unclear explanations of system mechanics; uneven participation; struggles with logic-flow questions.	Unprepared; cannot explain system flow or syscall interactions; unable to defend design decisions.